

ISAAC LEVINSKY

Rust Systems & Local AI — model training, inference, and shipped products

ilevinsky36@gmail.com | 386-576-6616 | github.com/IsaacLevinsky | mcmlv1.com

Tampa Bay Area, FL — Remote only, async-preferred • Full-time or contract

SUMMARY

Systems engineer who trains small language models from scratch and ships them into production alone — architecture, implementation, debugging, and release. Rust and C++/CUDA at the systems layer, Python/PyTorch for model work, C#/NET for native Android. All model training runs on a single 8-year-old Quadro GP100 workstation with no cloud spend.

SELECTED SYSTEMS

RustMind — LLM Training Stack Written From Scratch in Rust (Burn)

- Built the complete training path from first principles in Rust: model implementation, byte-level BPE tokenizer, corpus pipeline, validation design, and GPU training — no PyTorch anywhere in the training loop.
- Parameter ladder from 21M to 125M: proved generative training at 125M and a 72M encoder-decoder; trained the 21M encoder to production quality, where it runs in shipped software today.
- Root-caused a generation-collapse failure to non-stratified validation splits and over-memorized templated data, then rebuilt the corpus pipeline around document-aware chunking with explicit train/validation separation.
- Backend-as-type-parameter architecture (Rust generics over CUDA/CPU) — one codebase from GPU training to CPU-deployable inference.

rustmind-mini-embed-base — Public Pipeline Proof (MIT)

github.com/IsaacLevinsky/rustmind-mini-embed-base

- Python/PyTorch and Hugging Face release of the full training-to-inference pipeline: BPE tokenizer training (16k vocab) → MLM pretraining → ONNX export → INT8 dynamic quantization → local semantic search, with a reproducible CPU-only proof run and base checkpoint. Clone it and run it — I am glad to walk through any design decision in it.

LookingGlass — Zero-Dependency Rust Workspace Engine

- Compiled-only repo-analysis engine: project classification, language detection, dependency and reference graphing, automated audit reporting. **3,441 files in ~47ms**. Single binary — no Python, Node, or virtualenv runtime.
- Powers RustMind's corpus export with zero false positives across 30+ scanned repositories.

Orion — C++/CUDA Data Engine

- GPU-native pipeline converting structured input into compact binary streams, then GPU-accelerated cleaning, deduplication, validation, and scoring. **1.15M rows in 1.276s (~901K rows/sec)** on RTX 3060-class hardware.
- Deterministic CPU/GPU cross-validation kept acceleration inspectable. Related prototype work on genomics-scale sequence search with CUDA index and mismatch-scoring kernels.

Local Inference & Production AI

- **Bosun**: local-model routing across 8B–120B tiers via llama.cpp/llama-swap on owned GPU hardware; Android LLM workflows on LiteRT-LM (Gemma, LFM2) benchmarked to sub-2s stateless on-device response.
- **Quantization and hardware-aware deployment**: INT8 dynamic and Q4-class local models; identified FP16/BF16 numerical instability on Pascal-class hardware and reconfigured the runtime to avoid unstable training runs.
- **17 apps published** across Google Play, Amazon Appstore, and Samsung Galaxy Store — offline-first, native C#/NET Android with SQLite/WAL, on-device Whisper speech-to-text and ML Kit vision. No ads, accounts, or tracking.
- **Live AI in production on three surfaces**: a customer-service and sales chatbot on mcmlv1.com running on my own hardware, plus Google Play and Samsung Galaxy Store.

TECHNOLOGIES

Languages: Rust • Python • C++/CUDA • C#/NET • TypeScript/JavaScript • Go • Kotlin • SQL • Bash

ML & Inference: PyTorch • Hugging Face Transformers/Datasets/Tokenizers • Burn • ONNX Runtime • llama.cpp / llama-swap • LiteRT-LM • ML Kit • Whisper

Systems & Infrastructure: SQLite (WAL / FTS5 / BM25) • Docker • Linux • Cloudflare (Pages / Tunnel / R2) • Stripe • Git • AWS CLI

BACKGROUND & APPROACH

Self-directed engineering since 2021 — I learned by building and shipping real software, progressing from native Android and offline-first systems, to C++/CUDA performance work, to training language models from scratch. That path is why the work above spans hardware-level performance and applied ML rather than a single layer. Founded MCMLV1 LLC in January 2026 to publish it commercially.

Architecture-first: give me a constraint, a spec, or just a problem, and I design and deliver the whole system end to end.

Reliability-first — offline and local defaults, deterministic fallbacks, software built to run unattended for years. Strongest in async, output-measured environments.